Hazard Analysis
FPVR

Team 9, Software Mandems
John Cho
Ibrahim Tariq
Vikram Chandar
Sankalpa Chhetri Dhakal
Arjun Sasidaran

Table 1: Revision History

Date	**Developer(s)**	**Change**
Sept 30	Sankalpa Chhetri Dhakal	Completed Introduction
Oct 6	Sankalpa Chhetri Dhakal	Completed Scope and Purpose and System Boundaries
Oct 19, 2026	Arjun Sasidaran	Added FMEA (Section 5), user well-being hazards and SR21–SR24, and the roadmap (Section 7); restructured safety requirements into functional and non-functional subsections; marked multiplayer, user account, and database requirements as stretch goals

Contents

1 Introduction **1**

2 Scope and Purpose of Hazard Analysis **1**

3 System Boundaries and Components **1**

3.1 VR Headset 2

3.2 Controller Thumbsticks, Triggers, Buttons, and Input Connection 2

3.3 Host Computer 2

3.4 Unity Application 2

3.5 Drone Physics Simulation 3

3.6 VR Rendering and XR Subsystem 3

3.7 Training and Scoring System 3

3.8 User Interface 3

3.9 Data Storage 4

4 System Boundaries and Components **4**

5 Critical Assumptions **4**

6 Failure Mode and Effect Analysis **5**

7 Safety and Security Requirements **10**

7.1 Functional Requirements 10

7.2 Non-Functional Requirements 14

8 Roadmap **15**

8.1 Requirements Within the Capstone Timeline 15

8.2 Requirements Implemented in the Future 17

9 Appendix — Generative AI Use Disclosure **19**

9.1 AI Tools Used 19

9.2 How and Where Used 19
9.3 Verification of AI-Generated Content 20

10 Appendix — Reflection **22**

[You are free to modify this template. —SS]

1 Introduction

This document presents the Hazard Analysis for the VR-Based FPV Drone Operator Training Game. The purpose of the project is to provide users with a virtual environment where they can learn and practice FPV drone operation before flying a real drone. The system is intended to support both new users who are learning basic FPV controls and more experienced drone operators who want to practice manoeuvres, test their skills, or become familiar with different training scenarios in a low-risk environment.

For this project, a hazard is defined as a system condition, failure, or interaction that could result in harm or loss. This may include physical injury to the user, damage to equipment, loss of data, system failure, or incorrect training caused by unrealistic simulator behaviour. Although the system is primarily a software simulation, hazards may still arise from the use of VR hardware, software failures, inaccurate drone physics, or incorrect user interaction with the system.

2 Scope and Purpose of Hazard Analysis

3 System Boundaries and Components

The system boundary includes the software and hardware directly involved in running and interacting with the FPV drone training simulator. This includes the VR headset, input devices, host computer, Unity application, drone simulation, training functionality, user interface, and locally stored data. The user and their physical surroundings interact with the system and are considered when identifying hazards, but they are not components of the software system itself. Real FPV drones and real-world drone operation are outside the system boundary.

The system consists of several major software and hardware components. These components are considered independently during hazard identification because a failure in one component may affect the safety or correctness of the overall system.

3.1 VR Headset

The VR headset, expected to be a **Meta Quest 3S**, provides the user's visual representation of the simulated environment and tracks the position and orientation of the user's head.

Potential hazards associated with this component include loss of tracking, visual latency, incorrect orientation, poor headset fit, and the user becoming unaware of their physical surroundings.

3.2 Controller Thumbsticks, Triggers, Buttons, and Input Connection

Controller thumbsticks and triggers provide axis values that are mapped to drone throttle, pitch, roll, and yaw. Buttons provide discrete commands such as pause, restart, and respawn. Depending on the final implementation, these controls may be provided by VR controllers, a game controller, or an external FPV-style radio controller. The controller connection carries these values and button events to the application.

Failures may include thumbstick drift, where a centred stick reports an input and causes unintended drone movement; a stuck thumbstick or trigger, which maintains a control value after release; or a stuck or unresponsive button, which repeats or prevents a command. Incorrect axis mappings, inverted axis directions, or incorrect centre, range, or dead-zone calibration may produce unintended throttle or steering responses. A disconnected controller, dropped input updates, or delayed input delivery may cause loss of control, continued use of a stale control value, or a delayed response to a pilot command.

3.3 Host Computer

A computer executes the Unity application and associated simulation software.

Failures may include insufficient hardware performance, overheating, loss of power, driver issues, or resource exhaustion.

3.4 Unity Application

Unity provides the primary execution environment for the game.

This component is responsible for scene management, game logic, user interaction, rendering, and communication between major software modules.

3.5 Drone Physics Simulation

The drone physics subsystem models properties such as:

- thrust;
- gravity;
- acceleration;
- angular velocity;
- momentum;
- collisions; and
- drone response to pilot input.

Incorrect behaviour in this component represents an important hazard because users are expected to learn flying behaviour from the simulator.

3.6 VR Rendering and XR Subsystem

This subsystem renders the virtual environment and communicates with the VR headset.

Failures may result in visual artifacts, low frame rate, latency, incorrect camera movement, or loss of tracking.

3.7 Training and Scoring System

The training subsystem may contain tutorials, objectives, checkpoints, scoring metrics, crash detection, and user performance feedback.

Incorrect scoring or feedback could indicate that unsafe or incorrect flying behaviour is acceptable.

3.8 User Interface

The interface includes menus, settings, warnings, calibration options, pause controls, and training instructions.

Poor interface design could prevent the user from understanding warnings or exiting the simulation quickly.

3.9 Data Storage

The system may locally store information such as:

- user settings;
- controller configuration;
- training progress;
- scores; and
- application configuration.

Data corruption or unauthorized modification could affect system operation or training results.

4 System Boundaries and Components

[Dividing the system into components will help you brainstorm the hazards. You shouldn't do a full design of the components, just get a feel for the major ones. For projects that involve hardware, the components will typically include each individual piece of hardware. If your software will have a database, or an important library, these are also potential components. —SS]

5 Critical Assumptions

1. **Stable Internet Connection:** Users have access to a sufficiently stable internet connection for online features, including forms, multiplayer elements, and reviews.
   Rationale: Online features require reliable network connectivity to transmit user input, synchronize multiplayer gameplay, and submit or retrieve reviews. This assumption allows the system to be designed around the availability of network connectivity.

2. **Honest User Feedback:** Users provide honest gameplay reviews and feedback, and accurately report their experiences with the game.
   Rationale: Honest feedback helps ensure that reviews and user evaluations accurately represent gameplay experiences, allowing developers to identify issues and measure goals accurately without accounting for fake comments.

3. **Available Storage Capacity:** Users have sufficient available storage space on their systems to install the game and store any locally saved gameplay data.
   Rationale: The game requires storage for its installation files, course assets, and locally saved data. Insufficient storage may prevent installation, updates, or the saving of gameplay progress. It may require additional storage after progressing through levels such as save states and leaderboards, this storage should be available for the simulation to use.

4. **Basic Gaming Familiarity:** Users have basic familiarity with video games and standard user interface elements, including start screens, settings menus, and navigation controls.
   Rationale: This familiarity allows users to navigate the application and access gameplay features without requiring extensive introductory instruction, reducing the complexity of the user interface design.

5. **Suitable Hardware Performance:** Users have VR equipment that meets the minimum hardware requirements for running the game.
   Rationale: Meeting the minimum hardware requirements helps ensure that the system can maintain acceptable performance, provide a responsive VR experience, and execute the simulation as intended.

6. **Accurate Self-Reported Drone Experience:** Users who identify themselves as competent or experienced drone operators accurately represent their level of drone operating experience.
   Rationale: The system may rely on users' self-reported experience to understand their familiarity with drone operation and interpret their training results. Assuming these claims are accurate helps ensure that user classifications and performance evaluations are based on their stated level of experience.

6 Failure Mode and Effect Analysis

The following Failure Mode and Effect Analysis (FMEA) identifies how each major component of the system may fail, the effects and causes of each failure, and the actions taken to mitigate it. Each failure mode is traced to the safety and security requirements in Section 6 and, where applicable, to the functional requirements in the SRS. Each row is assigned a hazard reference (H) for traceability.

The SR column contains two types of identifiers. Identifiers beginning with SR or NFR followed by a number (e.g., SR5, NFR2) refer to the safety and security requirements defined in Section 6 of this document. Identifiers containing a component code (e.g., FR-FLT-04, FR-CHK-02, NFR-SAFE-01) refer to requirements defined in the FPVR Software Requirements Specification (SRS), a separate document.

Table 2: Failure Mode and Effect Analysis

Design Function	**Failure Mode**	**Effects of Failure**	**Causes of Failure**	**Recommended Action**	**SR**	**Ref**
Drone Flight Simulation	Drone responds incorrectly to flight inputs	Unexpected drone movement; user disorientation; inaccurate training experience	Invalid or out-of-range input values; errors in flight physics calculations	Validate and clamp controller input before applying it to the simulation; test flight physics against expected behaviour	SR5, NFR2, FR-FLT-02, FR-FLT-03	H1-1
	Collision is not detected, or a crash is triggered without contact	Incorrect crash count and score; user loses trust in the training feedback	Misconfigured colliders; drone passing through thin obstacles at high speed	Use continuous collision detection for the drone; add test cases for collisions at high speed and with each obstacle type	FR-FLT-04, FR-FLT-05	H1-2
VR Headset and Tracking	Headset disconnects or loses tracking during flight	Drone continues flying without reliable user input; user disorientation	Low headset battery; tracking interference; Quest Link cable disconnection	Suspend gameplay when the platform reports tracking loss or disconnection; restore access to the main menu on recovery	SR3, NFR4	H2-1
	Frame rate drops or rendering latency increases	Motion sickness, eye strain, or dizziness	Complex scenes exceeding hardware capability; unsupported hardware	Maintain the minimum frame rate on supported hardware; profile performance on each supported configuration	NFR1, NFR2, NFR3	H2-2
	User collides with physical objects while immersed	Physical injury to the user or damage to equipment	Obstacles in the play area; user unaware of physical surroundings	Display a safety notice before the first session; support seated play so users do not need to move physically	SR1, FR-UI-06	H2-3
	User cannot stop the session when feeling unwell	Worsening motion sickness or discomfort	No accessible pause or exit option during flight	Provide a pause and exit option at all times without requiring removal of the headset	SR2, NFR-SAFE-01	H2-4

Design Function	**Failure Mode**	**Effects of Failure**	**Causes of Failure**	**Recommended Action**	**SR**	**Ref**
Controller Input	Controller is not detected or disconnects	User cannot control the drone; flight continues without input	Controller not paired; low battery; unsupported controller	Verify that required hardware is detected before a flight session begins; suspend flight if the controller disconnects	SR4, FR-UI-03	H3-1
	Controller reports drifting or invalid values	Unintended drone movement and unnecessary crashes	Stick drift; hardware faults; incorrect input mapping	Apply dead zones and reject values outside supported input ranges	SR5	H3-2
Level and Checkpoint Management	Checkpoint progress advances out of order or fails to register	Users skip course sections or cannot complete a level	Incorrect checkpoint ordering; checkpoint trigger volumes too small	Advance progress only for the next required checkpoint; test each level for complete and out-of-order runs	FR-CHK-02, FR-CHK-03	H4-1
	Drone respawns at an incorrect or unsafe position	Repeated crashes immediately after respawning; user frustration	Incorrect respawn point; respawn point placed inside an obstacle	Respawn at the most recently completed checkpoint; verify every respawn point is clear of obstacles	FR-CHK-05	H4-2
Timing, Scoring, and Performance	Timers or collision count are not paused or reset correctly	Inaccurate score and performance rating	Timer state not updated on respawn, restart, or crash recovery	Pause the total timer during crash recovery; reset all performance data on a new attempt; add test cases for each transition	FR-PER-04, FR-PER-06, FR-LVL-07	H5-1
	Saved gameplay results are lost or corrupted	Loss of user progress and performance records	Application crash or network interruption during a save	Preserve previously saved records when errors occur; write records atomically	SR10	H5-2
Data Storage and Privacy	Stored gameplay records are accessed or modified by unauthorized parties	Exposure of user data; inaccurate performance records	Missing access controls; tampering with stored data	Restrict access to user-specific records; enforce authorization checks; verify record integrity on retrieval (Future; see Section 7)	SR8, SR14, SR17	H6-1

Design Function	Failure Mode	Effects of Failure	Causes of Failure	Recommended Action	SR	Ref
	User data is collected without the user's knowledge	Privacy concerns; loss of user trust	No disclosure of data collection practices	Disclose what data is collected, how it is used, and whether it is transmitted before collecting it	SR12	H6-2
	Malicious or malformed user input is processed	Corrupted records; injection attacks; unexpected behaviour	User-provided data not validated before processing or storage	Validate and sanitize all user-provided data	SR16	H6-3
	User account credentials are compromised	Unauthorized access to personal accounts and records	Weak authentication; plaintext password storage; brute-force attempts	Require authentication; hash passwords with a salted algorithm; limit repeated attempts; log security-relevant events (Future; see Section 7)	SR13, SR15, SR19, SR20	H6-4
Network and Multiplayer Services (Multiplayer: Stretch Goal)	Required external services are unavailable	Dependent features, such as intelligent feedback or multiplayer, fail; user unaware of the cause	Server outage; unstable internet connection	Verify connectivity before enabling dependent features; notify the user of multiplayer network interruptions (notification: Future; see Section 7)	SR6, SR11	H7-1
	Malformed or manipulated multiplayer messages are accepted	Inconsistent gameplay state; corrupted performance records	Untrusted client data; modified clients	Validate all data received from multiplayer clients before applying it (Future; see Section 7)	SR9	H7-2
	Network traffic or session credentials are intercepted or reused	Exposure of user data; unauthorized access to sessions	Unencrypted communication; session credentials not invalidated	Encrypt network communication; invalidate session credentials when a user leaves or a session expires (Future; see Section 7)	SR7, SR18	H7-3

Design Function	**Failure Mode**	**Effects of Failure**	**Causes of Failure**	**Recommended Action**	**SR**	**Ref**
Intelligent Feedback (Planned)	Feedback is unavailable or misleading	User receives incorrect guidance or no review	Analysis failure; insufficient flight data	Continue to provide the standard post-flight review when intelligent feedback fails	FR-AI-03, FR-REV-04	H8-1
User Well-being	User plays for extended periods without breaks	Eye strain, headaches, fatigue, or worsening motion sickness	Repeated attempts encourage continuous play; no reminders to rest	Display a break reminder after a defined period of continuous play; allow the user to pause or exit at any time	SR21, SR2	H9-1
	Visual effects trigger a photosensitive seizure	Seizure or injury to users with photosensitive epilepsy	Rapid flashing from crash effects, lighting, or high-contrast patterns	Limit flashing visual effects to safe frequencies; include a photosensitivity warning in the safety notice	SR22, SR1	H9-2
	Repeated crashes cause frustration or discouragement	Negative effect on user well-being and self-confidence; users abandon training	Difficulty increases too quickly; feedback focuses only on failures	Use neutral, constructive wording in crash feedback and results; suggest free-flight practice after repeated crashes in the same section	SR24, FR-LVL-01	H9-3
	User treats simulator results as readiness to fly a physical drone	Unsafe real-world drone operation, risking injury to people or damage to property	Simulated flight cannot reproduce every real-world condition, such as wind, signal loss, and battery limits	Remind users that simulator training complements real-world practice and applicable drone regulations	SR23	H9-4

7 Safety and Security Requirements

[Newly discovered requirements. These should also be added to the SRS. (A rationale design process how and why to fake it.) —SS]

7.1 Functional Requirements

SR1: The system shall provide users with a safety notice before their first VR session, reminding them to clear their physical play area of obstacles and remain aware of their surroundings.
Rationale: Users may collide with nearby objects while immersed in the virtual environment, potentially causing injury or damaging equipment.
Risk: Medium

SR2: The system shall allow users to pause or exit gameplay at any time during a VR session.
Rationale: Users may experience motion sickness, dizziness, or discomfort that requires them to stop playing immediately.
Risk: High

SR3: The system shall suspend active gameplay when the VR platform reports a headset disconnection or loss of tracking that prevents reliable interaction.
Rationale: Continuing gameplay when tracking is unreliable may cause unexpected interactions and increase user disorientation.
Risk: High

SR4: The system shall verify that required VR hardware is detected and initialized before allowing users to begin a flight session.
Rationale: Missing or improperly initialized hardware may prevent reliable controller input and compromise gameplay.
Risk: Medium

SR5: The system shall validate controller input values before applying them to the drone simulation and reject values outside supported input ranges.
Rationale: Invalid controller inputs may produce unintended drone movements or unpredictable simulation behaviour.
Risk: Medium

SR6: The system shall verify connectivity to required external services before enabling features that depend on them.
Rationale: Service connectivity failures may prevent multiplayer sessions or cause dependent features to behave incorrectly.
Risk: Medium

SR9: The system shall validate data received from multiplayer clients before accepting it for gameplay state updates or performance record storage.
Rationale: Malformed or manipulated network messages may cause inconsistent multiplayer behaviour or compromise recorded statistics.
Risk: High

SR11: The system shall notify users when a network interruption prevents access to multiplayer functionality.
Rationale: Users may otherwise be unaware that multiplayer functionality is unavailable or their session has been interrupted.
Risk: Low

SR12: The system shall disclose what gameplay data is collected, how it is used, and whether it is transmitted to external services before collecting user-associated data.
Rationale: Users need to understand how their gameplay information is handled to make informed decisions about their privacy.
Risk: Medium

SR13: The system shall require users to authenticate before accessing personal accounts or user-specific gameplay records, if account functionality is implemented.
Rationale: Authentication helps prevent unauthorized users from accessing personal information and saved progress.
Risk: High

SR14: The system shall enforce authorization checks before allowing users to view, modify, or delete stored gameplay records.
Rationale: Authorization prevents users from accessing or modifying records belonging to other users.
Risk: High

SR16: The system shall validate and sanitize user-provided data before processing or storing it.
Rationale: Improperly handled input may enable injection attacks, corrupt stored records, or cause unexpected system behaviour.
Risk: High

SR18: The system shall terminate or invalidate multiplayer session credentials when a user leaves a session or the session expires.
Rationale: Invalidating expired session credentials reduces the risk of unauthorized reuse or continued access to multiplayer sessions.
Risk: Medium

SR7: The system shall use encrypted communication when transmitting user data or multiplayer session information over a network.
Rationale: Unencrypted network traffic may expose user information or session data to unauthorized parties.
Risk: High

SR8: The system shall restrict access to stored user-specific gameplay records to the relevant user and authorized system components.
Rationale: Unauthorized access may expose private information or allow gameplay records to be modified.
Risk: Medium

SR10: The system shall preserve previously saved gameplay records when unexpected software errors or network interruptions occur.
Rationale: Unexpected failures may otherwise result in corrupted records or the loss of user progress.
Risk: Medium

SR15: The system shall securely store authentication credentials and shall not store passwords in plaintext. Passwords shall be protected using an appropriate salted password-hashing algorithm.
Rationale: Plaintext passwords may be exposed if stored data is accessed by an unauthorized party.
Risk: High

SR17: The system shall prevent unauthorized modification of stored gameplay records and verify data integrity when records are retrieved or updated.
Rationale: Tampered performance records may produce inaccurate results and undermine the reliability of user progress tracking.
Risk: Medium

SR19: The system shall limit repeated authentication attempts within a defined period.
Rationale: Limiting repeated attempts helps reduce automated password-guessing and brute-force attacks.
Risk: Medium

SR20: The system shall record security-relevant events, including failed authentication attempts and unauthorized access attempts, without unnecessarily logging sensitive user information.
Rationale: Security logs support the investigation of suspicious activity while minimizing the exposure of sensitive data.
Risk: Medium

SR21: The system shall display a reminder to take a break after 30 minutes of continuous play.
Rationale: Extended VR sessions can cause eye strain, headaches, and fatigue. A 30-minute interval is a common guideline for VR break reminders and gives users a regular opportunity to rest.
Risk: Medium

SR22: The system shall not display visual effects that flash more than three times in any one-second period.
Rationale: Rapid flashing can trigger seizures in users with photosensitive epilepsy. The three-flashes-per-second limit follows the WCAG 2.3.1 threshold for flashing content.
Risk: High

SR23: The system shall inform users that simulator training supports, but does not replace, real-world practice and applicable drone regulations.
Rationale: FPVR is designed to build flight skills, but simulated flight cannot reproduce every real-world condition, such as wind, signal loss, and battery limits. Users should continue practising in safe real-world conditions and follow

applicable regulations before operating a physical drone.
Risk: Medium

SR24: The system shall suggest free-flight practice after three consecutive crashes within the same checkpoint section.
Rationale: Repeated failures can discourage users and harm their confidence. Suggesting unscored practice gives users a low-pressure way to build skill before retrying the section.
Risk: Low

7.2 Non-Functional Requirements

NFR1: The system shall maintain a frame rate of at least 30 frames per second during standard single-player gameplay on the minimum supported VR hardware.
Rationale: Maintaining a consistent frame rate helps reduce visual discomfort and supports an immersive VR experience.
Risk: High

NFR2: The system shall respond to valid controller inputs within 50 milliseconds under normal operating conditions.
Rationale: Low input latency supports precise drone control and provides a more realistic flight simulation experience.
Risk: High

NFR3: The system shall operate correctly on all VR headsets, controllers, and operating system configurations explicitly listed as supported in the system documentation.
Rationale: Defining supported configurations ensures consistent operation and reduces hardware compatibility issues.
Risk: Medium

NFR4: The system shall restore access to the main menu within 10 seconds of recovering from a recoverable VR tracking or network interruption, provided that the required hardware and services are available.
Rationale: Prompt recovery minimizes disruption and reduces the need to restart the application.
Risk: Medium

8 Roadmap

The following roadmap outlines which of the safety and security requirements from Section 6 will be implemented within the capstone timeline and which will be implemented in the future.

8.1 Requirements Within the Capstone Timeline

- **SR1:** The system shall provide users with a safety notice before their first VR session.
  Justification: Physical injury is the most severe hazard in the system, and a safety notice is simple to implement before any user testing begins.
- **SR2:** The system shall allow users to pause or exit gameplay at any time.
  Justification: Users must be able to stop immediately if they feel unwell, so this is required before anyone plays the game.
- **SR3:** The system shall suspend active gameplay when a headset disconnection or loss of tracking is reported.
  Justification: Tracking loss can occur in every session, and continuing gameplay without reliable tracking directly causes user disorientation.
- **SR4:** The system shall verify that required VR hardware is initialized before a flight session begins.
  Justification: Every flight session depends on working hardware, so this check protects all other gameplay features from the start.
- **SR5:** The system shall validate controller input values before applying them to the drone simulation.
  Justification: Controller input drives the core flight simulation, so invalid values must be handled before flight features are built on top of it.
- **SR6:** The system shall verify connectivity to required external services before enabling dependent features.
  Justification: Checking connectivity is low effort and prevents features that depend on external services, such as the planned intelligent feedback, from failing without explanation during testing and demonstrations.
- **SR10:** The system shall preserve saved gameplay records during unexpected errors or network interruptions.
  Justification: Losing user progress during testing would undermine both the user experience and the evaluation of the system.
- **SR12:** The system shall disclose what gameplay data is collected and how it is used.
  Justification: Users must be informed before any data is collected, which includes participants in capstone user testing.

- **SR16:** The system shall validate and sanitize user-provided data before processing or storing it.
  Justification: Input validation is a basic safeguard that is far easier to include from the start than to add after data handling is built.

- **SR21:** The system shall display a break reminder after 30 minutes of continuous play.
  Justification: Eye strain and fatigue can affect any user, including participants in capstone user testing, and a timed reminder is simple to implement.

- **SR22:** The system shall not display visual effects that flash more than three times per second.
  Justification: Seizure risk is a severe hazard, and following the limit while visual effects are created avoids redesigning them later.

- **SR23:** The system shall inform users that simulator training supports, but does not replace, real-world practice and applicable drone regulations.
  Justification: The message requires little effort and should be in place before users begin relying on their training results.

- **SR24:** The system shall suggest free-flight practice after three consecutive crashes within the same checkpoint section.
  Justification: Free-flight mode is already part of the core system, so suggesting it only requires tracking consecutive crashes.

- **NFR1:** The system shall maintain at least 30 frames per second on the minimum supported VR hardware.
  Justification: Low frame rates cause motion sickness, so performance must be maintained throughout development rather than fixed at the end.

- **NFR2:** The system shall respond to valid controller inputs within 50 milliseconds.
  Justification: Responsive controls are central to the training experience and affect every flight session.

- **NFR3:** The system shall operate correctly on all explicitly supported hardware configurations.
  Justification: Defining supported hardware early focuses testing and prevents compatibility issues from appearing late in development.

- **NFR4:** The system shall restore access to the main menu within 10 seconds of a recoverable interruption.
  Justification: Interruptions are expected during normal VR use, and quick recovery avoids forcing users to restart the application.

8.2 Requirements Implemented in the Future

The following requirements depend on multiplayer functionality, user accounts, or database storage of gameplay records. These features are stretch goals of the project that may not be implemented within the capstone timeline, so the requirements will be implemented when the corresponding features are developed.

- **SR7:** The system shall use encrypted communication when transmitting multiplayer session information.
  Justification: Encryption only applies once multiplayer networking is implemented, which is a stretch goal.
- **SR8:** The system shall restrict access to stored user-specific gameplay records.
  Justification: Access restrictions only apply once user-specific records are stored, which depends on account functionality.
- **SR9:** The system shall validate data received from multiplayer clients.
  Justification: Validation of client data is only needed once multiplayer is implemented, and should be built alongside the multiplayer networking code.
- **SR11:** The system shall notify users when a network interruption prevents multiplayer access.
  Justification: This notification only applies to multiplayer features, which are a stretch goal.
- **SR13:** The system shall require users to authenticate before accessing personal accounts or records.
  Justification: Authentication is only needed once user accounts exist, which are a stretch goal.
- **SR14:** The system shall enforce authorization checks before stored gameplay records are viewed, modified, or deleted.
  Justification: Authorization depends on both authenticated users and stored records, so it follows the implementation of SR13.
- **SR15:** The system shall store passwords using a salted password-hashing algorithm.
  Justification: No passwords are stored until account functionality is implemented.
- **SR17:** The system shall prevent unauthorized modification of stored gameplay records and verify their integrity.
  Justification: Integrity verification depends on database storage of gameplay records, which is a stretch goal.

- **SR18:** The system shall invalidate multiplayer session credentials when a user leaves or a session expires.
  Justification: Session credentials only exist once multiplayer is implemented, and depend on user accounts; both are stretch goals.

- **SR19:** The system shall limit repeated authentication attempts within a defined period.
  Justification: Rate limiting only applies once an authentication system exists.

- **SR20:** The system shall record security-relevant events without logging unnecessary sensitive information.
  Justification: Most security events relate to authentication and record access, which depend on the user account and database stretch goals.

9 Appendix — Generative AI Use Disclosure

9.1 AI Tools Used

Claude Opus 5.5 and GPT Astra 6.1.

9.2 How and Where Used

1. **How were the AI tools used?**

   ChatGPT was used to brainstorm potential hazards for the FPVR drone flight simulation system, suggest safety and security requirements, and improve the clarity and organization of technical writing. It was also used to help identify potential risks that could be overlooked during manual analysis, including VR tracking loss, invalid controller inputs, network interruptions, unauthorized access, and corruption or loss of gameplay records. Additionally, ChatGPT assisted in developing rationales for proposed requirements and explaining why particular hazards could affect users or system functionality.

2. **Which parts of the document were affected by AI assistance?**

   AI assistance contributed to several sections of the Hazard Analysis document:

   - **Scope and Purpose of Hazard Analysis:** Assistance with organizing explanations of the purpose of identifying and mitigating hazards in the FPVR system.
   - **System Boundaries and Components:** Assistance with considering potential sources of hazards associated with VR hardware, controller inputs, networking, and gameplay data.
   - **Critical Assumptions:** Assistance with clarifying assumptions concerning internet connectivity, available storage, minimum hardware capabilities, gaming familiarity, and users' self-reported drone experience.
   - **Failure Mode and Effect Analysis:** Assistance with brainstorming potential failure scenarios, their effects, and possible mitigation strategies.
   - **Safety and Security Requirements:** Assistance with drafting and refining proposed requirements concerning VR safety, controller input validation, hardware initialization, network reliability, authentication, data privacy, and gameplay record integrity.
   - **Non-Functional Requirements:** Assistance with expressing proposed performance, responsiveness, compatibility, and recovery targets in measurable terms.

AI-generated suggestions were used as drafting and brainstorming aids rather than accepted automatically. The team remained responsible for reviewing the content and ensuring that the final document accurately reflected the FPVR project's scope, requirements, and design decisions.

9.3 Verification of AI-Generated Content

1. **How did the team check AI-generated or AI-assisted material before including it in the document?**

   The team treated AI-generated content as a starting point for brainstorming and drafting rather than accepting it without review. Suggestions were evaluated against the FPVR project's objectives, intended functionality, and existing requirements. For example, AI assistance was used to identify potential hazards associated with VR headset tracking loss, invalid controller inputs, and network interruptions. The team considered whether these scenarios were relevant to the system and whether the proposed mitigations addressed the associated risks. AI-assisted wording was also reviewed for clarity, consistency, and relevance before being included in the document.

2. **How did the team verify factual claims, technical assertions, requirements, calculations, and references?**

   Technical claims and proposed requirements were reviewed in the context of the FPVR system and the team's understanding of VR development and software engineering. For example, the proposed requirement to maintain at least 30 frames per second (NFR1) and the target of responding to controller inputs within 50 milliseconds (NFR2) were treated as design targets rather than confirmed system capabilities. These targets would need to be validated through performance testing on supported VR hardware. Similarly, proposed security requirements concerning encrypted communication, authentication, and gameplay record integrity were considered in relation to the system's planned networking and data-storage functionality.

   The team also reviewed requirement identifiers, rationales, and risk ratings for consistency and considered whether each proposed mitigation addressed its associated hazard. AI-generated suggestions were not treated as proof that a feature had been implemented or tested. Any factual claims, technical details, calculations, or external references used in the final document must be checked against reliable documentation or independent calculations where applicable. Where implementation details or test results were unavailable, the document distinguished proposed requirements from verified outcomes.

3. **Who is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document?**

The team is ultimately responsible for the accuracy, completeness, consistency, and appropriateness of the submitted Hazard Analysis, regardless of whether material was written by team members or generated with AI assistance. AI tools can support brainstorming, drafting, and identifying potential omissions, but their suggestions may be inaccurate, incomplete, or unsuitable for the project's scope. Therefore, the team must critically evaluate AI-assisted content and ensure that the final document accurately represents the FPVR system, its hazards, and its proposed mitigations.

10 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Vikram – Reflection

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?
4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?

1. **What went well while writing this deliverable?**

   The team was able to identify potential hazards associated with FPVR, our VR drone flight simulation game, and consider how they could affect users, gameplay, and system reliability. Examining the system component by component helped us identify potential failures involving VR hardware, controller inputs, the flight simulation, multiplayer functionality, and stored gameplay records. Translating these hazards into safety and security requirements also helped us define specific system behaviours that could be verified through testing.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

One challenge was distinguishing general software defects from hazards that could affect user safety, data integrity, or system reliability. For example, a VR tracking failure could affect both gameplay and user comfort, while a network interruption could cause multiplayer issues or lost progress. We addressed these challenges by examining the possible causes and consequences of each failure and identifying appropriate mitigation strategies. Another challenge was determining suitable risk classifications and writing requirements that were specific enough to test. We addressed this by providing rationales for each requirement and identifying system behaviours that could be verified during testing.

3. **How many of your requirements were inspired by speaking to your clients or their proxies, such as peers, stakeholders, and potential users?**

   Several requirements were informed by discussions with stakeholders and potential users, including drone enthusiasts and members of the McMaster MARS Rover team. For example, the requirement for responsive and realistic drone controls was motivated by the need to provide an authentic FPV flying experience that would be useful for both beginners and experienced drone operators. Another example is the requirement to provide multiple flight courses with varying levels of difficulty, which reflects the need to accommodate users with different levels of flying experience and allow them to progressively improve their skills. These discussions helped ensure that the system's requirements reflected the needs and expectations of its intended users.

4. **Which courses have you taken, or are currently taking, that will help your team succeed with the capstone project?**

   Several courses provide relevant knowledge for developing FPVR. Course 4HC3, Human Interface Design, will help the team design intuitive VR interactions, controller mappings, menus, and accessible interfaces. Its concepts will also support usability evaluation and improving user comfort.

   Course 4AA4, Control Systems, will help the team understand the relationship between controller inputs and simulated drone movement. Control systems concepts will support the development of responsive flight behaviour and the evaluation of the simulation's control model.

   Course 2AA4, Software Design, will help the team structure the application into modular components with clearly defined responsibilities and interfaces. This will be important when integrating VR interactions, flight physics, course management, multiplayer functionality, and performance evaluation.

   Additionally, programming and software engineering courses provide a foundation in object-oriented programming, requirements analysis, testing, debugging, and collaborative development. These skills will help the

team implement the simulation, verify its requirements, and integrate its components reliably.

5. **What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project?**

   The team will need to acquire knowledge and skills in several areas to complete FPVR successfully:

   - **VR development and hardware integration:** The team will need to learn Unity, C#, OpenXR, and the XR Interaction Toolkit to implement VR interactions, integrate supported headsets and controllers, and handle tracking information.
   - **Drone simulation and control systems:** The team will need to understand drone movement, flight dynamics, controller mappings, and how user inputs affect simulated flight behaviour.
   - **Performance optimization:** The team will need to learn how to profile Unity applications, identify rendering and processing bottlenecks, and optimize frame rate and input latency for a responsive VR experience.
   - **Software testing and user evaluation:** The team will need to develop test cases for gameplay, controller responsiveness, hardware compatibility, and recovery from interruptions. It will also need to evaluate usability, immersion, and user comfort through structured testing.
   - **Data analysis and training evaluation:** The team will need to learn how to record and interpret flight performance data, including completion times, crashes, control consistency, and unnecessary corrections. This knowledge will support performance summaries and the potential AI-based feedback feature.
   - **Version control and collaboration:** The team will need to use Git, feature branches, pull requests, and code reviews effectively. Consistent coding conventions and clearly defined component interfaces will help reduce integration problems.
   - **Technical documentation and communication:** The team will need to document implementation decisions, testing procedures, and design changes. Clear communication will also be necessary for coordinating work, resolving disagreements, and presenting the completed project.

   These areas can be distributed among team members according to their existing experience and responsibilities. Testing, documentation, and collaboration will remain shared responsibilities across the team.

6. **For each knowledge area and skill identified above, what are at least two approaches to acquiring the knowledge or mastering**

the skill? Which approaches will each team member pursue, and why?

The team will combine structured learning with practical implementation. Tutorials and documentation will provide foundational knowledge, while prototypes and testing will allow team members to apply that knowledge to FPVR.

- **VR development and hardware integration:** One approach is to complete official Unity, OpenXR, and XR Interaction Toolkit tutorials. A second approach is to develop a prototype that supports headset tracking, controller input, and basic VR interactions. The team member responsible for VR integration will pursue both approaches because tutorials provide a structured introduction, while prototyping reveals practical hardware and integration issues.
- **Drone simulation and control systems:** One approach is to review control systems course material and introductory resources on drone flight dynamics. A second approach is to implement a basic flight model and iteratively test how controller inputs affect movement. The team member responsible for flight simulation will combine theoretical study with experimentation to understand the underlying principles and tune the simulation's response.
- **Performance optimization:** One approach is to study Unity profiling tools and official VR optimization documentation. A second approach is to profile the application during gameplay, identify bottlenecks, and measure performance before and after changes. The team member responsible for performance will use both approaches to ensure that optimization decisions are based on measured results.
- **Software testing and user evaluation:** One approach is to study software testing techniques and usability evaluation methods. A second approach is to create test cases and conduct structured trials with representative users. The team member responsible for testing will combine these methods to identify both functional defects and usability issues.
- **Data analysis and training evaluation:** One approach is to study performance metrics and methods for evaluating control efficiency and consistency. A second approach is to build a prototype that records flight behaviour and generates feedback using measurable performance indicators. The team member responsible for performance evaluation will start with interpretable metrics before exploring more advanced AI-based approaches, allowing the evaluation feature to be validated incrementally.
- **Version control and collaboration:** One approach is to review Git documentation and collaborative development practices. A second approach is to apply feature branches, pull requests, and code

reviews in the project repository. All team members will practise these workflows because consistent collaboration is necessary to integrate independently developed components.

- **Technical documentation and communication:** One approach is to study examples of technical documentation and practise writing concise design descriptions and test reports. A second approach is to maintain project documentation and present progress during team meetings and project reviews. All team members will participate so that implementation decisions remain documented and project knowledge is shared.

The responsibilities described above are organized by technical area rather than by individual name. The team will assign specific owners based on members' experience, interests, and workload. This approach allows each member to develop a particular area of expertise while contributing to the team's overall progress.

7. **Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing the Hazard Analysis? For the latter risks, how did they come about?**

   Before completing the Hazard Analysis, the team had already considered risks related to VR performance, controller responsiveness, hardware compatibility, and user discomfort. These risks were closely related to the project's objective of providing an immersive, responsive, and usable drone flight simulation experience.

   The Hazard Analysis encouraged us to examine additional scenarios, including loss of VR tracking, invalid controller inputs, unexpected hardware disconnections, network interruptions, unauthorized access to gameplay records, and corruption or loss of saved progress. Some of these risks may have been recognized informally earlier, but the analysis helped us identify their causes and consequences more explicitly.

   These risks emerged by examining each system component and considering what could happen if it failed, received invalid data, or interacted incorrectly with another component. For example, examining controller inputs highlighted the possibility of unexpected drone movements. Examining multiplayer communication raised concerns about manipulated messages and inconsistent gameplay states. Finally, examining data storage highlighted the possibility of unauthorized access and the loss of saved records. This process demonstrated that risks extend beyond the simulation itself to include network communication, user information, and the reliability of stored data.

8. **Other than physical harm, list at least two other types of risk in software products. Why are they important to consider?**

   Several non-physical risks are relevant to FPVR.

Security and privacy risks: Unauthorized access to accounts or gameplay records could expose user information or allow stored results to be modified. These risks are important because users need to trust that their information is protected and that access to personal records is appropriately restricted.

Data integrity and reliability risks: Software errors, unexpected shutdowns, or network interruptions could corrupt gameplay records or cause users to lose progress. These risks are important because training evaluation and performance tracking depend on accurate and consistent data.

Performance and usability risks: Low frame rates, delayed controller responses, or confusing interactions could reduce immersion and make the simulation difficult to use effectively. These risks are especially important in VR because responsiveness and consistent visual feedback contribute to a comfortable and usable experience.

Compatibility risks: Differences between supported headsets, controllers, and operating system configurations could cause features to fail or behave inconsistently. Considering these risks helps the team define supported configurations and plan appropriate compatibility tests.

Project and integration risks: Poor coordination, incompatible components, or delays in integrating individual contributions could prevent the team from completing planned features within the capstone timeline. These risks are important because components that work independently may still fail when combined into the complete application.

Identifying these risks early allows the team to prioritize mitigations, make informed design decisions, and establish appropriate tests. Addressing them throughout development will help ensure that FPVR is reliable, secure, usable, and feasible to deliver within the available time.

Arjun Sasidaran

1. Organizing the FMEA by system component worked well. Working through the flight simulation, VR headset, controllers, checkpoints, scoring, data, network, and feedback components one at a time made it much easier to brainstorm failure modes than trying to list hazards for the whole system at once. Tracing every row to a safety requirement in Section 6 or a requirement in the SRS also helped me check coverage: any requirement without a matching failure mode, or any failure mode without a mitigation, stood out right away. Splitting the roadmap into capstone and future work, with a justification for each requirement, made our priorities much clearer than a single list would have.

2. The main pain point was deciding which safety and security requirements belong in the capstone timeline. Several requirements depend on multiplayer, user accounts, or database storage of flight records, which are all

stretch goals. Rather than removing them, I kept them so the hazards are still documented, moved them to the future section of the roadmap with a justification for each, and marked the related FMEA actions as future work. This keeps the analysis complete without committing the team to features that may not be built. Formatting was another pain point: the FMEA table was too wide for a portrait page, so I moved it to landscape pages, which then required extra work to keep the page numbers and column text readable.

3. Before this deliverable, we had already thought about the physical risks that are common to VR, such as motion sickness, colliding with objects in the room, and losing headset tracking, as well as basic performance risks like low frame rates and input latency. While completing the hazard analysis, we identified several less obvious risks. Going through each component led to gameplay logic failures, such as checkpoints registering out of order, unsafe respawn points, and incorrect timers affecting scores. Reviewing the hazard analysis checklist, which asks teams to think broadly about safety, led to user well-being risks: eye strain from long sessions, photosensitive seizures from flashing effects, frustration from repeated crashes, and users treating simulator results as readiness to fly a real drone. The last risk came from our own SRS, which already states that performance in FPVR is not proof of real-world flight ability.

4. One type of risk is security and privacy risk, such as unauthorized access to user data, intercepted network traffic, or compromised credentials. These matter because they can harm users even when the software works exactly as intended, and a breach can permanently damage user trust. A second type is psychological and well-being risk, such as frustration, discouragement, or software that encourages unhealthy amounts of use. This is especially important for games and training tools used by the general public, since their design directly shapes how users feel and behave. A third type is data integrity risk, where results or progress are lost or recorded incorrectly. In a training tool like FPVR, inaccurate scores or lost progress would give users misleading feedback about their skills.